SECURITY FUNDAMENTALS AND ASSESSMENT REPORT

THE UNSECURE PWA COMPANY

UNSECURE PWA Security Report – Beau Bastock

Document details

Assessment

Date	<Month YYYY>

Prepared by IN_SYD Software Engineering Security

Assessor name	Beau Bastock

Prepared for The Unsecure PWA Company

Contact name	Mr Dwyer

Adapted from Australian Signals Directorate's Australian Cyber Security Centre (2022).

Contents

1. Executive summary....4
2. Introduction....5
3. Benefits of developing secure software....7
4. Influence of end users on secure design features....8
5. Developing secure code....9
6. Security features incorporated into software....12
7. ‘Security by design’ and ‘Privacy by design’ approaches....14
8. Testing and evaluating security and resilience....17
9. Strategies for managing security of programming code....19
10. Defensive data input handling....21
11. Efficient execution for the user....23
12. Security and privacy vulnerabilities in the unsecure PWA....25
13. Other common security and privacy vulnerabilities....40
14. References....41
15. Security report appendix....42

1. Executive summary

1.1 Briefly outline the purpose of the report.

This report was prepared by Beau Bastock on behalf of IN_SYD Software Engineering Security, commissioned by The Unsecure PWA Company to assess the security and privacy of their progressive web application. The application was evaluated during its testing and debugging phase of the software development lifecycle. The assessment identified and resolved four security vulnerabilities: broken authentication, cross-site scripting (XSS), cross-site request forgery (CSRF), and invalid forwarding and redirecting. Additionally, a SQL injection vulnerability was identified and resolved. For each vulnerability, the report documents an explanation of the issue, the business risk if exploited, a technical description of how it exists in the source code, how it was identified, and how it was fixed with supporting code evidence. The report also covers broader security concepts including secure software development principles, cryptography, sandboxing, privacy by design, security by design, defensive data input handling, and strategies for managing code security.

2. Introduction

2.1 Overview of ‘The Unsecure PWA Company’.

The client, ‘The Unsecure PWA Company’, has engaged us as a software engineering security specialist to provide expert advice on the security and privacy of their application.

This progressive web app is currently in the testing and debugging phase of the software development lifecycle and can be accessed here: Unsecure PWA.

2.2 Explain the importance of secure software architecture in today’s digital landscape.

Secure software architecture can be broken down word by word. In today’s digital landscape secure refers to the state of being protected from unauthorized access, cyber threats and vulnerabilities. Secured software ensures that data, systems, services remain trustworthy by maintaining confidentiality, integrity, and availability which are core principles of the CIA triad. Software architecture refers to high level structure and design of a system, when security is embedded into this stage of the software development life cycle it prevents vulnerabilities from being built into the system from the start, developers can limit the impact of potential breaches and reduce the systems attack surface. In a world of cloud computing, APIs and constantly evolving cyber threats, secure software architecture is essential for protecting user data, ensuring system reliability, meeting legal and ethical obligations, and maintaining long-term software sustainability.

2.3 Describe the steps you took to identify security vulnerabilities, resolve those vulnerabilities and how you validated they had been resolved

To identify security vulnerabilities in the unsecure PWA, two key methods were used. A manual code review of the application source files including ‘main.py’, ‘user_management.py’ and the HTML template files was performed. Reading through each file to identify suspicious patterns such as unvalidated user input, missing authentication controls and insecure database queries. This was supported by manual penetration testing of the running application, where common attack inputs were submitted through the login and feedback forms to confirm vulnerabilities could be exploited. For example HTML markup was injected into the feedback form to confirm XSS, and a single character password was used during signup to confirm, the absence of password strength enforcement. To resolve the identified vulnerabilities, targeted fixes were applied directly to the source files. This included removing unvalidated redirect blocks from main.py, adding CSRF token protection using the flask WTF library, applying html.escape() to feedback output user_management.py, implementing bcrypt password hashing and salting the database, adding rate limiting via flask limiter on the login route.

To validate that each fix was successful, the application was retested after each change was made. For example, after applying hashing to the passwords prior to entry on the database a new account was made with a unique password, upon review of the database any passwords entered after the fix were now hashed and unreadable. After applying html.escape() HTML markup instantly changed to be shown as plaintext not executable javascript and after implementing password validation weak passwords were entered to confirm the form blocked submission before data even reached the server.

3. Benefits of developing secure software

3.1 Explain the benefit of developing secure software in relation to “data protection”.

Developing secure software is essential as it is one of the strongest ways to protect data. Assuring vulnerabilities are never opportunities for attackers. When security is implemented proactively it drastically reduces the chances of data being accessed, altered or stolen. This protects users’ information such as name, address, financial records or other sensitive data. Strong security measures such as encryption (two-way private keys or hashing) ensure even if an attempt is made to breach the data remains confidential, accurate and available to authorised users. Secure software directly supports data protection by minimizing risks, maintaining user trust, and ensuring enterprises meet legal and ethical responsibilities regarding safeguarding personal information.

3.2 Explain the benefit of developing secure software in relation to “minimising cyber-attacks and vulnerabilities”.

Developing secure software is essential for minimising cyber-attacks and vulnerabilities because embedding security into the software development life cycle ensures that threats are addressed before they can be exploited. Secure development practices such as input validation, authentication and authorisation and error handling prevent common vulnerabilities such as injection attacks and unauthorised access. Techniques like threat modelling and risk management help identify weaknesses early, allowing teams to implement controls that maintain the confidentiality, integrity, and availability of data. Availability is particularly important in the context of attacks such as Distributed Denial of Service (DDoS), which aim to overwhelm a system and bring it offline entirely. By designing software with resilience in mind, including rate limiting and load handling, systems can remain operational even under attack. In a digital environment where cyber-attacks are increasingly sophisticated, developing secure software not only protects users and organisations but also ensures long term system reliability, compliance and resilience.

4. Influence of end users on secure design features

4.1 Analysis of how user capabilities and experiences affect security features

Explain how the capabilities and experience of end users influence the secure design features of software.

End user design focuses on creating products, systems or services tailored specifically to the need, behaviours and preferences of the people who use them. A user focused design will support users in maintaining the security of an application. The capabilities of end users impact the design features of software by influencing the user interface, accessibility options, customisation preferences and security settings. Therefore, consider that different users will have different needs & skills such as a less technical person who may require a simplified control view to manage their privacy and security settings while a more experienced user may prefer granular control over permissions and configurations. Designing with this range of users in mind ensures that security features are accessible and usable for everyone, reducing the likelihood of users making mistakes that compromise their own security. If security features are too complex or difficult to understand, users will ignore or bypass them, weakening the overall security of the application regardless of how well those features are technically implemented.

5. Developing secure code

5.1 Fundamental Software Design Security Concepts

Explanation of key concepts:

Table 1 – fundamental software design security concepts

Concept	Explanation
Confidentiality	Confidentiality ensures that personal and sensitive information is encrypted and security stored to prevent unauthorised access. This is important to protect user credentials and personal data from being accessed by unauthorised parties, maintaining privacy and trust in the system. Confidentiality can be ensured by encrypting web traffic using HTTPS which prevent interception of web traffic, encrypting passwords using hashing (protects passwords if the database is hacked.) and apply authorisation controls to ensure only appropriate people can access personal and sensitive information.
Integrity	Integrity verifies data accuracy, consistency and reliability to prevent unauthorised tampering or alteration of data throughout its lifecycle. This is important as it enables high quality decision making from quality assured data. This improves operational efficiency, regulatory compliance and customer trust. Without its organisations could face financial lost and severe reputational damage and with healthcare industries significant risks to safety. Data integrity can be ensured by implementing checksums, digital signatures and hash functions to detect and prevent data tampering. Use encryption to protect data integrity during transmission.
Availability	Availability ensures that system services and resources are accessible to authorised users when needed. This is important to prevent downtime, ensure continuity of

Concept	Explanation
	services and maintain user satisfaction. This can be ensured by implementing a design with redundancy, load balancing and disaster recovery mechanisms. As well as regularly monitoring system performance and address potential issues proactively. Availability is maintained through prevention of cyber-attacks such as DDOS (distributed denial of service) attacks. This is a malicious attempt to disrupt a server, service or network traffic by overwhelming it with a flood internet traffic from multiple compromised services. Prevention against DDOS attacks maintain PWA uptime.
Authentication	Authentication verifies the identity of users, devices or systems before granting access to the system. This is essential as it prevents unauthorized access and ensures users are who they claim to be. This is implemented using strong authentication systems such as passwords (strong policies such as requiring a max length, mix of characters and symbols), multi factor authentication systems (Password + SMS code), 2FA (Through an Auth App) and use secure protocols for user authentication (HTTPS).
Authorisation	Enforces access control to determine user, application or device permissions and restrict access based on roles. It determines the actions (read, write delete) a user can perform on resources. This is important to prevent unauthorised activities and protects sensitive data from being accessed by unauthorised users. This can be prevented by implementing role-based access control (RBAC) restricting based on user's role rather than individual identity, or attribute-based access control more flexible approach granting access based on individual user attributes (ABAC). As well as regularly review and update user permissions based on users' roles and responsibilities.

Concept	Explanation
Accountability	The principle that security vulnerabilities are clearly defined, assigned and enforced across an organisation ensuring that any individual (from board to staff) own their roles in protecting data. Implementing logging and auditing mechanisms to track user activities and security events. This is important as it establishes traceability responsibilities in case of security incidents or breaches. This can be done by maintaining logs of user actions, access attempts, and security events. Regularly reviewing audit logs and investigate any suspicious activities ideally have system alerts triggered for suspicious activities.

6. Security features incorporated into software

6.1 Provide an overview of the following security features

Table 2 – security features

Security feature	Application
Data protection	Data protection is the process of safeguarding sensitive information from corruption, compromise or loss, ensuring both security and availability. Such as secure password storage (hashing of passwords), Encrypt data at rest (within database) and in transit (between client device and web server) and data retention policies to only keep personal and sensitive data for as long as required.
Security measures	Security measures are actions, procedures, or devices such as locks, surveillance, or encryption implemented to prevent unauthorized access, theft, damage or threats to physical property, data and information systems. Protection against common vulnerabilities such as XSS (by sanitizing data), CSRF (CSRF tokens) and broken authentication (strong passwords and two factor authentication.)
Privacy protection	Privacy protection is safeguarding of personal information, identifying personal or sensitive data from unauthorised access. Privacy protection in Australia follows the APP (Australian Privacy Principles). Enterprises should document and implement their own privacy policy, obtain user consent for how you capture and use personal data, and lastly apply the Australian Privacy Principles which are 13 principles that govern the standards, rights and obligations around the collection use of personal information.
Regulatory compliance	Regulatory compliance is the mandatory practice of adhering to laws, regulations and industry standards. Complying with the laws within your region such as the GDPR for the EU or the case of the PWA the APP (Australian privacy principles). The system needs to align its security measures with regulatory requirements and apply the APP throughout the system to prevent legal scandals, fines and loss of consumer trust.

7. ‘Security by design’ and ‘Privacy by design’ approaches

7.1 Contribution of cryptography and sandboxing

Explain the role of cryptography in data security to the client, ‘The Unsecure PWA company’.

Cryptography is the practice of developing and using coded algorithms to protect and obscure information so that it may only be read by those with the permission and ability to decrypt it. Password hashing involves transforming a plaintext password into a fixed-length string of characters using a hashing algorithm. This one-way function ensures that even if someone gains unauthorized access to the hashed password, they cannot easily revert it to the original password. In relation to the PWA this appears as the storing of passwords in the database. The current PWA stores plain-text passwords into the database leaving the security vulnerability of unauthorised access to private data. If a user was able to retrieve the contents of the database, they would gain access to all user passwords compromising data security and user privacy. Failing to store hashed passwords violates the Australian Privacy Principles (APP) leading to large legal and monetary issues. Without hashing passwords, the PWA loses client trust, enterprise reputation and faces legal repercussion. Furthermore, the PWA doesn’t apply HTTPS to secure web traffic and prevent data being sent in plaintext across http and apply authentication to ensure a user is connected to the correct website. HTTP stores in plaintext meaning anyone intercepting the data can clearly see information such as passwords, credit cards or session cookies reducing client trust and breaking APP leading to legal and monetary issues.

Explain how sandboxing limits exposure to vulnerabilities.

Sandboxing is the technique of isolating untrusted applications or processes from the rest of the system, limiting potential damage to the host system. Providing a safe space for running untrusted code or applications securely. Key aspects include isolation and containment, malware analysis allowing safe detonation of malware, viruses or phishing attachments to observe their actions and threat detection for identifying unknown threats. Sandboxing provides benefits of a safe testing environment without jeopardizing system integrity.

Summarise security by design for the client.

Security by design is a software development approach that aims to develop secure software through a proactive approach that embeds security controls into every step of the development process. Leaving security as an afterthought typically results in insecure software and higher development costs when security elements need to be retrofitted at a later stage. A proactive security by design approach considers ways to reduce both the impact and likelihood of security vulnerabilities throughout the entire software development life cycle.

In practice this means integrating security-focused activities into each of the fundamental software development steps. During requirements definition, security requirements are established alongside functional ones, such as ensuring the software conforms to the Australian Privacy Principles, which govern how personal data must be collected, stored and handled. During design, threat modelling is used to identify potential attack surfaces before any code is written. During development, secure coding practices such as input validation, parameterised queries and proper authentication are applied. During testing, security testing including penetration testing and code review is performed to catch vulnerabilities before deployment. During maintenance, software is monitored and patched in response to newly discovered threats.

By treating security as a core requirement rather than an optional layer added at the end, security by design reduces the cost of fixes, minimises the risk of breaches, and produces software that is more resilient and trustworthy for users and organisations alike.

7.2 Privacy by design approach

Provide the client with the principles of proactive privacy measures.

Proactive privacy measures follow the “privacy by design” and “security by design” principles. They follow the principles of proactive not reactive which means that security fundamentals are included throughout each step of the development process. Leaving security as an afterthought typically results in one or both of; insecure software and higher software development costs for security elements built at a late state. A proactive approach will consider ways to reduce the impact (mitigate) of security vulnerabilities (encrypting passwords to reduce the impact of a database hack) and ways to reduce the likelihood of security vulnerabilities (ensuring a web application is developed to prevent XSS).

Explain the importance of embedding privacy into software design to the client.

Privacy by design is proactively embedding into technology and business processes rather than an afterthought. Preventing data breaches, legal compliance (Australian Privacy Principles) and builds customer trust by following high standards. Regulatory compliance is a key requirement for the APP preventing heavy penalties, legal disputes and financial loss. Customers are more likely to engage with services that clearly protect their data which enhances confidence in products or services. Furthermore, embedding a privacy by design approach is more cost effective as integrating privacy at the beginning of the development cycle is much cheaper than retrofitting systems to fix privacy later.

8. Testing and evaluating security and resilience

8.1 Provide the client, 'The Unsecure PWA Company', with methods for identifying vulnerabilities and creating resilience.

Table 3 – methods for identifying vulnerabilities and creating resilience

Method	Description
Hardening systems	Systems hardening is a proactive, continuous process designed to reduce organizations attack surface and eliminate vulnerabilities across IT infrastructure, servers, networks, applications and firmware. This includes resolving found vulnerabilities within a system to make the system secure and applying security patches for supporting software and infrastructure. Vulnerabilities can be found by performing strategies such as code reviews, SAST, DAST, vulnerability assessments and penetration testing.
Handling breaches	Handling breaches refers to the structured, strategic process of identifying, responding to, mitigating and recovering from an incident where confidential or personal data has been stolen, exposed or accessed without authorization. Enterprises must be transparent and promptly communicate with customers after an incident has occurred and fulfil their responsibilities from the Australian Privacy Act.
Maintaining business continuity	Maintaining business continuity refers to the system remaining available for use. Systems can become unavailable for use due to: - Cyber attacks - Infrastructure failure - Insufficient infrastructure to handle user volume Systems should be hardened to prevent cyber-attacks as well as sized to handle predicted peak usage of users, then tested to confirm. Frequent backups of data should

Method	Description
	be made to enable quick disaster recovery with no to minimal loss of data.
Disaster recovery plan	A disaster recovery plan is a documented, structured approach used to restore IT infrastructure, data and operations after a cyberattack. Systems should have a disaster recovery plan which needs to be designed and tested to enable prompt recovery from failures of key infrastructure (e.g. servers in a data centre)

9. Strategies for managing security of programming code

Explain to the client the strategies used by software developers to manage the security of programming code.

Table 4 – strategies for managing security

Strategy	Explanation
Code Reviews	Code reviews are a security focused review of the code by an experienced developer who DID NOT write the code. This person looks through the code to detect any vulnerabilities. This identifies vulnerabilities in the custom source code of the project it won't identify problems with supporting software or servers. Code reviews improve software quality, enhance team knowledge and reduce long term maintenance costs. Code reviews happen early in development where it is cheaper to fix issues allowing for prevention of vulnerabilities before live deployment and real user data.
Static application security testing (SAST)	SAST is an automated tool-based check of the source code (or compiled binaries) for security vulnerabilities. During this process the application will not be running. As this process is automated it can be cheap to execute only dealing with upfront purchase of the tool, however false positives can be identified. SAST reduces remediation costs, ensures compliance with security standards and increases overall code quality through its comprehensive scanning of the entire codebase.
Dynamic application security testing (DAST)	DAST is a security method that tests running web applications for vulnerabilities by simulating external malicious attacks. It identifies runtime flaws like SQL injection and XSS without accessing source code. This is often an automated tool but can also involve manual testing of the runtime application. DAST provides lower false positives rates, the ability to detect configuration, session and authentication flaws in real time run during pre-production to ensure applications are secure before they go live.
Vulnerability assessment	Vulnerability assessment is a systematic, proactive process of identifying, classifying and reporting security weaknesses in software, hardware and

Strategy	Explanation
	networks.
Penetration testing	Penetration Testing is an authorized, simulated cyberattack on a computer system, network, or web application to identify security vulnerabilities before malicious hackers can exploit them. This is run by a specialist team (white hat hackers) typically using DAST tools. These tests can happen without informing development/support. Its purpose is to identify security gaps, evaluate compliance (APP), test employee awareness and determine the impact of a successful breach.

10. Defensive data input handling

Provide the client with code snippets that demonstrate best practices for input validation, sanitisation, and error handling.

Defensive data input handling is a proactive secure coding practice that treats all incoming data as untrustworthy, ensuring only, valid, safe and well-informed data enters an application. Defensive data handling relates to both security and usability.

From a security perspective sanitization is critical to ensure that input data is cleansed of the code. The best practice is to sanitize prior to storage AND on retrieval which protects against XSS. It is critical that data sanitization occurs on the backend layer as hackers can bypass front end data sanitization. Input validation is important at ensuring the integrity of the data.

From a usability perspective input validation and user-friendly error handling supports users to correctly use the system and enter data correctly. Examples include valid birthdays not letting anyone set a birthdate in the future.

These code snippets demonstrate best practices for sanitisation in prevention of XSS on the unsecure PWA:

```
def listFeedback():
    con = sql.connect("database_files/database.db")
    cur = con.cursor()
    data = cur.execute("SELECT * FROM feedback").fetchall()
    con.close()
    f = open("templates/partials/success_feedback.html", "w")
    for row in data:
        f.write("<p>\n")
        f.write(f"{html.escape(row[1])}\n")
        f.write("</p>\n")
    f.close()
```

```python
def insertFeedback(feedback):
    con = sql.connect("database_files/database.db")
    cur = con.cursor()
    sanitised_feedback = escape(feedback)
    cur.execute("INSERT INTO feedback (feedback) VALUES (?)", (sanitised_feedback,))
    con.commit()
    con.close()
```

Beau Bastock, last month • Add files via upload …

11. Efficient execution for the user

Inform the client and users of the strategies for effective memory, session, and exception management.

Table 5 – strategies for effective memory, session, and exception management

Strategy	Explanation
Memory management	Memory management is an operating system process that controls and coordinates a computers main memory, allocating it to running applications and freeing it up when no longer needed otherwise causing slower program execution Efficient memory management balances quick data retrieval and overall RAM usage. This is especially important for online applications which may use memory for each concurrent user session. A memory leak occurs when a program use memory but then fails / forgets to release it. Most modern operating systems are more efficient at preventing memory leaks if the system doesn't manage its memory well. Most modern programming languages have features to prevent / minimize memory leaks, however care is still needed by programmers.
Session management	Session management allows a user (authenticated or as a guest) to perform secure, authorized tracking of user interactions with a web application, allowing stateful communication over stateless HTTP whilst: - Keeping their data private from other users - Not needing to reauthenticate For example, for a session that involves a user logging

Strategy	Explanation
	into their online back account, then performing a series of actions such as viewing account, view transactions, etc. each new transaction is a unique HTTP request before the user then logs off. The webserver needs to manage the user's session so that each new HTTP request relates to the ongoing session, so the user only needs to authenticate once and so the user's information is private from other sessions. Furthermore, multiple people will be logged in simultaneously the server needs to manage multiple concurrent sessions.
Exception management	Exception (unexpected errors) management is a proactive business strategy focusing on identifying, analysing, and resolving significant deviations from planned performance, standards, or norms. Exceptions should be caught and handled by the system. End users should be provided with user friendly messages and an option to retry the action and advice on how to seek help. End users should not be given a technical message, especially crucial information on software architecture. Logs and alerts should be made technical to the support staff to allow debugging.

12. Security and privacy vulnerabilities in the unsecure PWA

For each of the following vulnerabilities:

- Explain the vulnerability
- What is the risk (impact to business) if the vulnerability is exploited
- Provide a technical description of the vulnerability including an explanation of how it exists in the source code of the unsecure PWA
- Describe how you identified the vulnerability in the unsecure PWA application (e.g. by performing a code review)
- Describe how the vulnerability was fixed, including images of the updated source code

Table 6 – vulnerabilities

Vulnerability	Detailed explanation and resolution
Broken authentication	1. Explanation Broken authentication covers any weaknesses in how an application handles user identity, login and credentials. When authentication is poorly implemented, attackers can get into accounts without ever knowing the correct password. They can guess repeatedly with no consequences, target users who chose weak passwords, or measure response times to figure out which usernames are real. Any of these is enough to cause serious damage. 2. Risk (Impact) An attacker with unlimited login attempts can run automated tools that try thousands of common passwords against any account, and with no lockout in place nothing prevents this. Users who set weak passwords like “password” or “123456” are the easiest targets and without strength requirements the application has no way to prevent those passwords from being registered in the first place. The narrow timing window in the login response leaks whether a username exists, letting an attacker build a list of valid accounts before they even start guessing. If any account is compromised the attacker gets full access

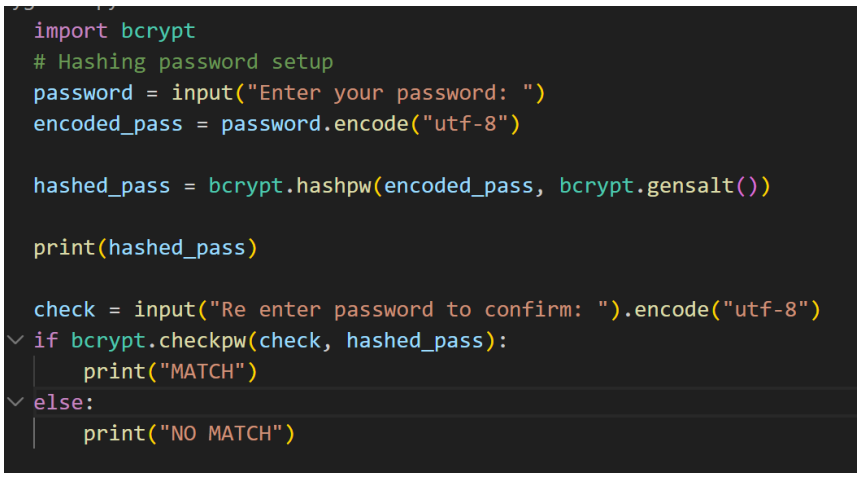
Vulnerability	Detailed explanation and resolution
	to that users' data. Beyond the direct impact, a breach carries reputational damage and potential liability under Australian Privacy legislation. 3. Technical Description There are four authentication weaknesses in the application. The first is the absence of password strength requirements. The signup form in 'signup.html' accepts any value in the password field, so a single character password goes through without complaint. Weak passwords are far easier to crack through brute force or dictionary attacks. The second is no rate limiting on the login route. In 'main.py', the home() function passes submitted credentials directly to a retrieveUsers() on every POST request with nothing to restrict how many times that can happen. An attacker can hammer the endpoint to programmatically with no consequence. The third is a timing vulnerability in 'user_management.py'. The artificial delay is set to random.randint(80,90) / 100, a range of only 10 milliseconds. That consistency means response time differences between valid and invalid usernames are potentially measurable, letting an attacker enumerate real accounts before attempting any passwords. The fourth relates to how passwords were originally stored. Without hashing, a database breach would expose every user's password in plain text. Passwords were also stored without a salt, leaving them vulnerable to rainbow table attacks where an attacker uses a precomputed lookup table to reverse common password hashes. Hashing passwords and applying salt prevents the use of rainbow table attacks. 4. How It was Identified

Vulnerability	Detailed explanation and resolution
	The vulnerabilities were identified through a combination of code review and manual penetration testing. A code review of 'user_management.py' revealed that passwords were originally inserted into the database as plain text with no hashing or salt applied, leaving them fully exposed in the event of a database breach. The same file also revealed the artificial timing delay was to a range of only 80-90 milliseconds, narrow enough that response time differences between valid and invalid username lookups could be measurable by an attacker. Manual penetration testing of the signup form confirmed that no password strength requirements were enforced, by entering a single character of the password "1" into the field and successfully registering an account, it was clear the application had no mechanism to reject weak passwords before they reach the database. 5. How the vulnerability was fixed Three fixes were implemented to address the broken authentication weaknesses. The first fix added client-side password strength validation to the password field in signup.html, using a pattern attribute. The pattern enforces a minimum of 8 characters and requires at least one uppercase letter, one lowercase letter and one number. The browser displays a descriptive message to the user if their password does not meet the requirements and the form will not submit until it does. This means a weak password is never sent to the server or stored in the database. The second fix added rate limiting to the login route in 'main.py' using the Flask-Limiter library. A limiter instance was initialised using 'get_remote_address' to identify clients by their IP address, and a @limiter.limit("5 per minute") decorator was applied to the home()

Vulnerability	Detailed explanation and resolution

	route. Any IP address that exceeds 5 login attempts within a minute automatically receives an error 429 Too Many Requests response and is blocked until the window resets, making automated brute force attacks impractical.
	
	The third fix widened the artificial timing delay in 'user_management.py' from a narrow range of 80 to 90 milliseconds to a much wider range of 50 to 200 milliseconds. The original 10 millisecond window was consistent enough that response time differences between valid and invalid username lookups could be measurable by an attacker. The wider 150 range makes those timing differences indistinguishable, preventing username enumeration through response time analysis.
	
	As an additional fix that was implemented separately, passwords are now hashed and salted using bcrypt before being stored in the database. This means a database breach no longer exposes plaintext password, and the unique salt applied to each password makes precomputed rainbow table attacks ineffective.
	
	LISTED FIXES BELOW
	
	`time.sleep(random.randint(50, 200) / 1000)`
	`con.close()`
	`return password_matches`
	
	`<div class="input__wrapper">`
	`<input type="password" name="password" placeholder="Password"`
	`class="input__field" pattern="(?=.*\d)(?=.*[a-z])(?=.*[A-Z]).{8,}"`
	`title="Password must be at least 8 characters and include an uppercase letter,`
	`lowercase letter, and a number" required>`
	
	`# LIMITER`
	`limiter = Limiter(get_remote_address, app=app, default_limits=[])`
	
	`@app.route("/", methods=["POST", "GET"])`
	`@limiter.limit("5 per minute")`
	`def home():`

Vulnerability	Detailed explanation and resolution

	
	import bcrypt
	# Hashing password setup
	password = input("Enter your password: ")
	encoded_pass = password.encode("utf-8")
	
	hashed_pass = bcrypt.hashpw(encoded_pass, bcrypt.gensalt())
	
	print(hashed_pass)
	
	check = input("Re enter password to confirm: ").encode("utf-8")
	if bcrypt.checkpw(check, hashed_pass):
	print("MATCH")
	else:
	print("NO MATCH")

Cross-site scripting (XSS)	1. Explanation Cross site scripting (XSS) is a vulnerability where an attacker can inject malicious HTML or JavaScript code into a web application, which is then rendered and executed in the browsers of other users who view that content. This occurs when a user supplied input is displayed back on a page without being sanitized or encoded first. There are different types of XSS, in this case it is stored XSS (also called persistent XSS), meaning the malicious input is saved to the database and execute every time any user loads the page containing that content. 2. Risk (Impact) If the XSS vulnerability were exploited, the impact on the business could be severe. Because any HTML or JavaScript submitted through the feedback form is stored in the database and rendered directly on the page, an attacker could inject malicious scripts that execute automatically in the browsers of every user who visits that page. This could be used to redirect users to phishing or malware sites, steal session cookies to hijack authenticated accounts, or deface the applications interface to damage the business's credibility. Even in a less severe case, injected content such as misleading buttons or links could deceive users into taking actions, they did not intent. The persistent nature of stored XSS makes it particularly damaging compared to a one-time attack, the malicious content remains active

Vulnerability	Detailed explanation and resolution
	and affecting users until it is manually removed from the database. 3. Technical Description The vulnerability exists in the user_management.py inside the listFeedback() function. When feedback is submitted, main.py passes it directly into insertFeedback() which stores it in the database with no validation. When listFeedback() is then called, it reads every feedback entry from the database and writes them directly into success_feedback.html. Because no encoding or escaping is applied at this point whatever was stored in the database is written into the HTML file as is. This means submitted HTML tags are written as real HTML markup rather than plain text. When success_feedback.html is later included in success.html via Jinja2's {% include %} the browser receives and renders it as functioning HTML. So a submitted button, link or script tag appears and executes on the page for every user who loads it. An attacker could go beyond injecting a simple button instead submit a <script> tag to run Javascript, for example reading document cookies and sending session data to a server they control. 4. How it was identified The vulnerability was found through manual testing of the feedback form. Rather than submitting normal text, HTML markup was entered into the feedback field and submitted. On reload, the injected HTML rendered and functioned in the browser. Confirming the app was not encoding user input before displaying it. A follow-up code review of 'user_management.py' revealed that the listFeedback() function writes feedback values from the database directly into 'success_feedback.html' using an f-string with no encoding or escaping applied, meaning any HTML submitted as feedback is written raw into the file and interpreted by the browser as markup. 5. How the vulnerability was fixed the fix was applied in user_management.py inside the listFeedback() function. This function is responsible for reading all feedback entries

Vulnerability	Detailed explanation and resolution

	from the database and writing them directly into success_feedback.html. meaning the XSS vulnerability existed at the point of the file writing. The fix involved importing pythons built in 'html' module and wrapping each feedback value with html.escape() before it gets written to the file. This converts dangerous characters such as "<" and ">" into their safe HTML equivalents. So, a submitted <button> tag is a written to the file as <button@gt; and displayed as harmless plaintext in the browser than a functioning HTML element. The change was a single line replacing f.write(f"{row[1]}\n") with f.write(f"{html.escape(row[1])}\n"). And adding import html at the top of user_management.py. From now on any feedback containing injected HTML or JavaScript is now neutralised before it ever reaches the file, meaning it cannot execute in any user's browser regardless of how it was stored in the database.
	
	BEFORE
	
	`def listFeedback():`
	`  con = sql.connect("database_files/database.db")`
	`  cur = con.cursor()`
	`  data = cur.execute("SELECT * FROM feedback").fetchall()`
	`  con.close()`
	`  f = open("templates/partials/success_feedback.html", "w")`
	`  for row in data:`
	`  f.write("<p>\n")`
	`  f.write(f"{row[1]}\n")`
	`  f.write("</p>\n")`
	`  f.close()`
	
	AFTER

Vulnerability	Detailed explanation and resolution

	``python
	def listFeedback():
	    con = sql.connect("database_files/database.db")
	    cur = con.cursor()
	    data = cur.execute("SELECT * FROM feedback").fetchall()
	    con.close()
	    f = open("templates/partials/success_feedback.html", "w")
	    for row in data:
	        f.write("<p>\n")
	        f.write(f"{html.escape(row[1])}\n")
	        f.write("</p>\n")
	    f.close()
	``
	
	``python
	def insertFeedback(feedback):
	    con = sql.connect("database_files/database.db")
	    cur = con.cursor()
	    sanitised_feedback = escape(feedback)
	    cur.execute("INSERT INTO feedback (feedback) VALUES (?)", (sanitised_feedback,))
	    con.commit()
	    con.close()
	``
	
	1. Explanation
	
	Cross site request forgery (CSRF) is a vulnerability where an attacker tricks an authenticated user to unknowingly submitting a malicious request to a web application they are already logged into. Because the browser automatically includes session cookies with every request the application cannot distinguish between a legitimate user action and one forged by an attacker. The attacker hijacks the user's authentication session to perform actions on their behalf without their knowledge.
	
	2. Risk (Impact)
	
	If the CSRF vulnerability were exploited, the consequences for the business would be significant. An attacker could trick an authenticated user into unknowingly submitting forged requests, allowing them to perform unauthorised actions such as submitting fake feedback, creating fraudulent accounts, or manipulating existing user data without the victim's knowledge. If the attack were directed at a password or account change function it could result in full account takeover, locking legitimate users out of their own accounts. Beyond

Cross-site request forgery (CSRF)	

Vulnerability	Detailed explanation and resolution
	the direct technical impact, the business would suffer reputation damage as users lose confidence in the security of the platform. Additionally, under the Australian Privacy Principles if users' data is altered or exposed because of a vulnerability, the business may face regulatory scrutiny or legal liability 3. Technical Description The vulnerability exists because the flask application has no CSRF token validation on any of its forms. When a form is submitted via POST, flask only checks that the required form fields are present. It does not verify that the request was originated from the application's own pages. If a logged in user visits a malicious page their browser silently submits the form to the legitimate application and the application will accept it as there is no token to verify the requests origin. In 'main.py' the POST handlers in home(), signup() and addFeedback() all read directly from request.form with no origin or token check. 4. How it was identified The vulnerability was identified through a manual code review of main.py and the HTML template files. Examining the POST route handlers revealed that form submissions are processed using only "request. Form["field"]" with no accompanying token validation. Reviewing the HTML form templates confirmed there were no CSRF tokens present. The lack of CSRF middleware or per request token generation in the PWA confirms it is fully vulnerable. 5. How The Vulnerability was Fixed The fix was implemented using the flask_wtf librarys CSRFProtect extension following three steps: First, the flask_wtf library was installed via pip. Then main.py was updated by importing CSRF protect and os, setting a SECRET_KEY using os.urandom(32) to cryptographically sign tokens, initialising

Vulnerability	Detailed explanation and resolution

	CSRFProtect(app) immediately after the app is created. This enforces token validation across all POST requests. Finally, a hidden CSRF token field was added to each HTML form using Flasks 'csrf_token()' template function. This embeds a unique server generated token into every form at the time it is rendered. When the form is submitted, Flask checks that the token matches what was originally issued. Any forged requests from an external site will not posses a valid token and will automatically be rejected.
	
	BEFORE
	
	``python
	import user_management as dbHandler
	
	# Code snippet for logging a message
	# app.logger.critical("message")
	
	app = Flask(__name__)
	# Enable CORS to allow cross-origin requests (needed for CSRF demo in Codespaces)
	CORS(app)
	``
	``html
	<h1>Signup Form</h1>
	<form action="/signup.html" method="POST" class="box">
	  <div class="input__wrapper">
	  <input type="text" name="username" placeholder="Username" class="input__field">
	  </div>
	``
	
	AFTER
	
	``python
	from flask_wtf import CSRFProtect
	import os
	
	# Code snippet for logging a message
	# app.logger.critical("message")
	
	app = Flask(__name__)
	app.config["SECRET_KEY"] = os.urandom(32)
	csrf = CSRFProtect(app)
	``
	``html
	<h1>Signup Form</h1>
	<form action="/signup.html" method="POST" class="box">>
	  <input type="hidden" name="csrf_token" value="{{ csrf_token() }}"/>
	  <div class="input__wrapper">>
	``
	
	1. Explanation

Invalid forwarding and	Invalid forwarding and redirecting are a vulnerability where a web

Vulnerability	Detailed explanation and resolution

redirecting	application accepts user-controlled input to determine where to redirect the user, without validating that the destination is safe or trusted. Because the redirect destination is not checked, an attacker can manipulate it to send victims to any external website of their choice. This is dangerous because the initial link appears to come from a legitimate, trusted domain, making the attack harder for victims to detect.
	
	2. Risk (impact)
	
	If exploited the vulnerability can have serious consequences for the business. An attacker can send users a link in a phishing scam that looks like it belongs to a legitimate site (https://trusteddomain.com??url=https://evilsite.com) users trust that domain, clicks it and is silently redirected to a fake login page designed to steal credentials. Since the attacker is using the businesses own domain as the base for its attacks it causes users to lose trust when they discover that they were redirected to a malicious page damaging the PWA's reputation. If the redirect is embedded in an authentication or password reset flow, attacks can intercept tokens or session data by redirecting the response to a server they control. Furthermore, the PWA will face legal and regulatory liability, under the APP if user data is compromised because of the vulnerability the business may face legal and monetary penalties.
	
	3. Technical Description
	
	The vulnerability exists in three separate route handlers within 'main.py', home(), signup() and addFeedback(). Each one contains the following block:
	
	`if request.method == "GET" and request.args.get("url"):`
	`  url = request.args.get("url", "")`
	`  return redirect(url, code=302)`
	
	Request.args.get("url") retrieves the value of the url query directly from the incoming HTTP Get request. This value is entirely user controlled

Vulnerability	Detailed explanation and resolution

	which anyone can append ?url=https://evilsite.com to the address and that value is passed without any validation directly into the redirect() function which issues a 302 HTTP redirect to the attacks URL. Since there was no allowlist, domain check, or any form of sanitisation applied before the redirect it executes and an attacker has full control over where the application sends the user.
	
	4. How it was identified
	
	The vulnerability was identified through a manual code review of 'main.py'. While reading through each route handler the same pattern occurred three times. In home(), signup() and addFeedback() functions. Each case the application was reading a 'url' parameter from the GET request query string using request.args.get("url") and passing it directly to redirect() with no validation logic present. Tracing the data flow confirmed the value originates entirely from user supplied input and is never checked against any list of permitted destinations before the redirect is used. A clear example of invalid forwarding / redirecting.
	
	5. How it was fixed
	
	The fix is to remove all three redirect blocks from 'main.py', as the redirection functionally served no essential purpose in the applications core logic. Removing it eliminates the attacks surface. The affected routes continue to function correctly for their intended purposes.
	
	BEFORE
	
	``
	@app.route("/success.html", methods=["POST", "GET", "PUT", "PATCH", "DELETE"]
	def addFeedback():
	    if request.method == "GET" and request.args.get("url"):
	        url = request.args.get("url", "")
	        return redirect(url, code=302)
	    if request.method == "POST":
	        feedback = request.form["feedback"]
	        dbHandler.insertFeedback(feedback)
	        dbHandler.listFeedback()
	        return render_template("/success.html", state=True, value="Back")
	    else:
	        dbHandler.listFeedback()
	        return render_template("/success.html", state=True, value="Back")
	``

Vulnerability	Detailed explanation and resolution

	AFTER
	
	``python
	@app.route("/success.html", methods=["POST", "GET", "PUT", "PATCH", "DELETE"])
	def addFeedback():
	    if request.method == "POST":
	        feedback = request.form["feedback"]
	        dbHandler.insertFeedback(feedback)
	        dbHandler.listFeedback()
	        return render_template("/success.html", state=True, value="Back")
	    else:
	        dbHandler.listFeedback()
	        return render_template("/success.html", state=True, value="Back")
	``
	
	If forwarding or redirecting were genuinely required by the application, it should be implemented using an allowlist of permitted internal paths like the images below:
	
	``python
	ALLOWED_REDIRECTS = ['/dashboard', '/profile', '/home']
	
	if request.method == "GET" and request.args.get("url"):
	    url = request.args.get("url", "")
	    if url in ALLOWED_REDIRECTS:
	        return redirect(url, code=302)
	    else:
	        return redirect('/home', code=302)
	``
	
	This ensures the redirect destination is never taken at face value from user input. Only known internal paths are permitted. Any unrecognised destination causes the application to fallback to a safe default instead.

	A SQL injection attack consists of inserting or "injecting" SQL code via the input data from the client to the application. The application will then execute a modified version of the SQL that will have negative consequences. **What is the risk (impact to business) if the vulnerability is exploited** **The SQL injection manipulated queries can result in the following**

Vulnerability	Detailed explanation and resolution
	issues.

Security Issue	Risk (Impact to business)
Manipulated queries resulting in sensitive data being revealed	Data privacy breaches
Manipulated queries inappropriately updating data	Data integrity issues
Manipulated queries altering the database, such as dropping tables	System availability (i.e. the system would crash)
Manipulated queries allowing an unauthorised user to logon as someone else	Authorisation issues which then result in the unauthorised user gaining access to private data and being able to perform unauthorised actions.

This can result in reputational damage and loss of trust of users. Serious breachers of the Australian Privacy Principles may result in legal issues for the business.

Provide a technical description of the vulnerability including an explanation of how it exists in the source code of the unsecure PWA

The issue exists as the SQL query is constructed with string

Vulnerability	Detailed explanation and resolution

	concatenation rather than using query parameters.
	
	Specifically, the following line in the user_management.py file
	
	`cur.execute(f"SELECT * FROM users WHERE username = '{username}' AND passw`
	
	Describe how you identified the vulnerability in the unsecure PWA application (e.g. by performing a code review)
	
	The issue was identified through manual testing of the running application (DAST) which tested common SQL injection flaws.
	
	Using the password hi' or '1' = '1 in the password field allowed a user to login without knowing the valid password.
	
	The SQL interface code was then examined through a code review to look for vulnerable code.
	
	Describe how the vulnerability was fixed, including images of the updated source code
	
	The vulnerability was fixed by making a change to use query parameters instead of string concatenation.
	
	`cur.execute('SELECT * FROM users WHERE username == ? AND password == ?',`

13. Other common security and privacy vulnerabilities

Provide an explanation of these common security and privacy vulnerabilities.

Vulnerability	Outline
Race conditions	A race condition is a software flaw where the security of the system depends on uncontrollable timing or sequence of controllable events, such as multiple threads accessing shared data updating / using a common resource. Race condition vulnerabilities can be hard to identify usually requiring white box pen testing with DAST tools to generate high volume of HTTP requests. An example includes a user using a banking system to withdraw and deposit simultaneously to violate the integrity of their balance data. This can be prevented by ensuring shared resources are accessed by only one process or thread at a time.
File attacks	A file attack is where a threat uses certain file types (DOCX, XLSX, or PDF) to entice users into open a document and follow a malicious link. If malicious code is embedded in the file, it will be run when opened. For example, phishing emails that contain malicious attachments and file injection attacks to upload harmful code. Prevention involves securing systems against malicious code with strict upload controls, constant scanning for malware, phishing detection and prevention.
Side channel attacks	In computer security, a side-channel attack is a type of security exploit that leverages information inadvertently leaked by a system, such as timing, power consumption, or electromagnetic or acoustic emissions. To gain unauthorized access to sensitive information. For example, if the time taken to login with a valid username responds slower than an invalid username, a hacker would be able to determine valid usernames. To prevent this a randomized delay can be implemented within the system to prevent consistent results for hackers to determine valid and invalid requests.

14. References

Australian Privacy Principles https://www.oaic.gov.au/privacy/australian-privacy-principles

OWASP XSS https://owasp.org/www-community/attacks/xss

OWASP CSRF https://owasp.org/www-community/attacks/csrf

OWASP Broken Authentication https://owasp.org/www-project-top-ten/2017/A2_2017-Broken_Authentication

OWASP Unvalidated Redirects https://owasp.org/www-project-top-ten/2017/A10_2017-Unvalidated_Redirects_and_Forwards

Australian Cyber Security Centre https://www.cyber.gov.au

Flask-WTF CSRF Documentation https://flask-wtf.readthedocs.io/en/stable/csrf

Python html.escape Documentation https://docs.python.org/3/library/html.html

bcrypt Documentation https://pypi.org/project/bcrypt

Student resources in the GitHub repo

15. Security report appendix

Any additional information or supporting documents.

Additional Vulnerability Identified: Malicious Third-Party Script Inclusion

During a manual code review of layout.html a suspicious external script tag was identified. The script was loading a JavaScript file from an unrecognised external domain with no relation to the legitimate Bootstrap project. Because layout.html is the base template used across every page of the application, any malicious code loaded through this script would execute in every user's browser across the entire PWA. This could be used to steal user data, redirect users to malicious sites or compromise user accounts, causing significant reputational damage to the business and potential breaches of the Australian Privacy Principles.

The fix applied was to remove the script tag entirely from layout.html.